I

I'm excited to share my recent data analyst project, where I conducted a comprehensive analysis of retail giant's sales data!

Data Analysis Report

By-Madhusudhana

Q1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1. Data type of all columns in the "customers" table.

```
SELECT
  column_name,
  data_type
FROM
  `ecommerce-394317.target`.INFORMATION_SCHEMA.COLUMNS
WHERE
  table_name = 'customers'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

PERSONAL HISTORY | PROJECT HISTORY

Key findings: The data type of customers table is as shown above. From Schema it is inferred that customer_id is the primary key of customers table.

2. Get the time range between which the orders were placed.

```
SELECT
 MIN(order_purchase_timestamp)AS first_order,
 MAX(order_purchase_timestamp)AS last_order
 FROM `target.orders`
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	first_order	last_order
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Key findings: The data set contains orders placed for the period 2016 to 2018. The first order was placed on 04Sep2016 and the last one on 17Oct2018.

3. C. Count the number of Cities and States in our dataset.

```
SELECT
 COUNT(DISTINCT(customer_city)) AS no_of_unique_cities,
 COUNT(DISTINCT(customer_state)) AS no_of_unique_states
FROM `target.customers`
```

Query results

JOB INFORMATION | RESULTS | JSON

Row	no_of_unique_cities	no_of_unique_states
1	4119	27

Key findings: The operations were successfully carried out in all 27 states and across 4119 cities of Brazil.

II. In-depth Exploration:

A. Is there a growing trend in the no. of orders placed over the past years?

```
SELECT
 *
FROM (SELECT
 EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
 EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
 COUNT(order_id) AS no_of_orders
FROM `target.orders`
GROUP BY EXTRACT(MONTH FROM order_purchase_timestamp), EXTRACT(YEAR FROM
order_purchase_timestamp))
ORDER BY year,month
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	month	year	no_of_orders
1	9	2016	4
2	10	2016	324
3	12	2016	1
4	1	2017	800
5	2	2017	1780
6	3	2017	2682
7	4	2017	2404
8	5	2017	3700
9	6	2017	3245
10	7	2017	4026

Key findings: We can see a gradual increase in number of orders along some particular months.

B. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

```
SELECT
 *
FROM (SELECT
 EXTRACT(MONTH FROM order_purchase_timestamp) AS month,
 COUNT(DISTINCT order_id) AS no_of_orders
FROM `target.orders`
GROUP BY EXTRACT(MONTH FROM order_purchase_timestamp))
ORDER BY month
```

Query results

SAVE RESULTS EXP

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART PREVIEW | EXECUTION GRAPH

Row	month	no_of_orders
1	1	8069
2	2	8508
3	3	9893
4	4	9343
5	5	10573
6	6	9412
7	7	10318
8	8	10843
9	9	4305
10	10	4959

Results per page: 50 1 – 12 of 12

Key findings: There is particular bimonthly trend wherein consecutive two months have almost the same number of orders.

C. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

First approach

```
SELECT
 order_id,
 customer_id,
 CASE
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 0 AND 6 THEN "Dawn"
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 7 AND 12 THEN "Mornings"
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 13 AND 18 THEN "Afternoon"
  ELSE "Night"
  END AS time_of_day
FROM `target.orders`
ORDER BY order_purchase_timestamp
LIMIT 10
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRA

Row	order_id	customer_id	time_of_day
1	2e7a8482f6fb09756ca50c10d...	08c5351a6aca1c1589a38f244...	Night
2	e5fa5a7210941f7d56d0208e4...	683c54fc24d40ee9f8a6fc179f...	Dawn
3	809a282bbd5dbcabb6f2f724fc...	622e13439d6b5a0b486c4356...	Afternoon
4	bfbd0f9bdef84302105ad712d...	86dc2ffce2dfff336de2f386a78...	Mornings
5	71303d7e93b399f5bcd537d12...	b106b360fe2ef8849fbbd056f7...	Night
6	3b697a20d9e427646d925679...	355077684019f7f60a031656b...	Mornings
7	be5bc2f0da14d8071e2d45451...	7ec40b22510fdbea1b08921dd...	Afternoon
8	65d1e226dfaeb8cdc42f66542...	70fc57eeae292675927697fe0...	Night
9	a41c8759fbe7aab36ea07e038...	6f989332712d3222b6571b1cf...	Night
10	d207cc272675637bfed0062ed...	b8cf418e97ae795672d326288...	Night

Key findings: From this approach we can know which order was placed when.

Second appraoach:

```
SELECT
 COUNT(order_id) AS num_of_orders,
 CASE
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 0 AND 6 THEN "Dawn"
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 7 AND 12 THEN "Mornings"
  WHEN (EXTRACT(HOUR FROM order_purchase_timestamp))
  BETWEEN 13 AND 18 THEN "Afternoon"
  ELSE "Night"
  END AS time_of_day
```

```
FROM `target.orders`
GROUP BY time_of_day
ORDER BY num_of_orders DESC
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	num_of_orders	time_of_day
1	38135	Afternoon
2	28331	Night
3	27733	Mornings
4	5242	Dawn

Key findings: From this approach we can find the numb er of orders at the particular time of day. We can infer that a majority of orders were placed in the afternoon then followed by night and then by mornings and then lastly dawn.

III. Evolution of E-commerce orders in the Brazil region:

A. Get the month on month no. of orders placed in each state.

```
First approach
SELECT
 DISTINCT
 EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
 EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
 c.customer_state,
 COUNT(DISTINCT order_id) AS num_of_orders
FROM `target.customers` AS c
INNER JOIN `target.orders` AS o
ON c.customer_id = o.customer_id
GROUP BY c.customer_state,month,year
ORDER BY year,month, c.customer_state
LIMIT 10
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	month	year	customer_state	num_of_orders
1	9	2016	RS	1
2	9	2016	RR	1
3	9	2016	SP	2
4	10	2016	SP	113
5	10	2016	MG	40
6	10	2016	GO	9
7	10	2016	CE	8
8	10	2016	SC	11
9	10	2016	RJ	56
10	10	2016	RS	24

Key findings: This output shows the distribution of orders across states and month and year when it was placed.

B. How are the customers distributed across all the states?

```
SELECT
 DISTINCT
 customer_state,
 COUNT(DISTINCT customer_unique_id) AS num_of_customers
FROM `target.customers`
GROUP BY customer_state
ORDER BY customer_state
LIMIT 10
```

Query results

SAVE RESULTS | EXPLORE DATA

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART PREVIEW

Row	customer_state	num_of_customers
1	AC	77
2	AL	401
3	AM	143
4	AP	67
5	BA	3277
6	CE	1313
7	DF	2075
8	ES	1964
9	GO	1952
10	MA	726

Key findings: This output shown the number of customers spread across each states of Brazil.

IV. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

A. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

```
SELECT
 DISTINCT
 SUM(p.payment_value) OVER (PARTITION BY EXTRACT(YEAR FROM
o.order_purchase_timestamp),EXTRACT(MONTH FROM o.order_purchase_timestamp)) AS
monthly_total_2017,
 EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
 EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
FROM `target.payments` AS p
JOIN `target.orders` AS o
ON p.order_id = o.order_id
WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017)
GROUP BY year,month,o.order_purchase_timestamp,p.payment_value
ORDER BY year, month
```

B. Calculate the Total & Average value of order price for each state.

```
SELECT
 c.customer_state,
 ROUND(SUM(p.payment_value),1) AS state_total,
 ROUND(AVG(p.payment_value),1) AS avg_state_total
FROM `target.payments` AS p
JOIN `target.orders` AS o
ON p.order_id = o.order_id
JOIN `target.customers` AS c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY c.customer_state
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | E

Row	customer_state	state_total	avg_state_total
1	AC	19680.6	234.3
2	AL	96962.1	227.1
3	AM	27966.9	181.6
4	AP	16262.8	232.3
5	BA	616645.8	170.8
6	CE	279464.0	199.9
7	DF	355141.1	161.1
8	ES	325967.5	154.7
9	GO	350092.3	165.8
10	MA	152523.0	198.9

Key findings: The output shows the total order value and average order value spread across each state.

D. Calculate the Total & Average value of order freight for each state

```
SELECT
  DISTINCT
  c.customer_state,
  SUM(oi.price*oi.freight_value) OVER (PARTITION BY c.customer_state) AS
total_freight_value ,
  AVG(oi.price*oi.freight_value) OVER (PARTITION BY c.customer_state) AS
avg_freight_value
FROM `target.customers` AS c
JOIN `target.orders` AS o
ON c.customer_id = o.customer_id
JOIN `target.order_items` AS oi
ON o.order_id = oi.order_id
WHERE o.order_status = "delivered"
ORDER by total_freight_value DESC,c.customer_state
LIMIT 10
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHAR

Row	customer_state	total_freight_value	avg_freight_value
1	SP	113030682.3466	2433.488682970...
2	RJ	51027764.7787	3607.987327914...
3	MG	46182854.7453	3575.631367706...
4	RS	22143652.8617	3609.985794212...
5	PR	19653461.634	3479.104555496...
6	BA	18687662.3912	5074.032688351...
7	SC	17268617.2418	4214.941967732...
8	PE	11828539.6433	6774.650425715...
9	CE	10027479.9814	7031.893395091...
10	DF	9749839.7024	4140.059321613...

Key findings: The output shows the total freight value and average freight value spread across each state.

V. Analysis based on sales, freight and delivery time.

A. Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

```
SELECT
 DISTINCT
 order_id,
 DATE_DIFF(order_delivered_customer_date,order_purchase_timestamp, DAY) AS
time_to_deliver,
 DATE_DIFF(order_estimated_delivery_date,order_delivered_customer_date, DAY) AS
diff_estimated_delivery
FROM `target.orders`
WHERE order_status = "delivered"
ORDER BY order_id
LIMIT 10
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUT

Row	order_id	time_to_deliver	diff_estimated_delivery
1	00010242fe8c5a6d1ba2dd792...	7	8
2	00018f77f2f0320c557190d7a1...	16	2
3	000229ec398224ef6ca0657da...	7	13
4	00024acbcdf0a6daa1e931b03...	6	5
5	00042b26cf59d7ce69dfabb4e...	25	15
6	00048cc3ae777c65dbb7d2a06...	6	14
7	00054e8431b9d7675808bcb8...	8	16
8	000576fe39319847cbb9d288c...	5	15
9	0005a1a1728c9d785b8e2b08...	9	0
10	0005f50442cb953dcd1d21e1f...	2	18

Key findings: The output shows the time to deliver and diff estimated delivery time for each order id.

B. Find out the top 5 states with the highest & lowest average freight value.

i) Query for top-5 highest average freight value in ascending order.

```
SELECT
  DISTINCT
  c.customer_state,
  AVG(oi.price*oi.freight_value) OVER (PARTITION BY c.customer_state) AS
avg_freight_value
FROM `target.customers` AS c
JOIN `target.orders` AS o
ON c.customer_id = o.customer_id
JOIN `target.order_items` AS oi
ON o.order_id = oi.order_id
ORDER by avg_freight_value ASC
LIMIT 5 OFFSET 22
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAI

Row	customer_state	avg_freight_value
1	AC	8176.334146739...
2	AL	8394.420707207...
3	RO	9291.884083453...
4	TO	9500.006578730...
5	PB	11892.82567026...

ii) Query for top-5 lowest average freight value in ascending order.

```
SELECT
  DISTINCT
  c.customer_state,
  AVG(oi.price*oi.freight_value) OVER (PARTITION BY c.customer_state) AS
avg_freight_value
FROM `target.customers` AS c
JOIN `target.orders` AS o
ON c.customer_id = o.customer_id
JOIN `target.order_items` AS oi
ON o.order_id = oi.order_id
ORDER by avg_freight_value ASC
LIMIT 5
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTIO

Row	customer_state	avg_freight_value
1	SP	2450.252187150...
2	PR	3527.413892630...
3	MG	3602.542219559...
4	RJ	3650.452331332...
5	RS	3811.455463464...

C. Find out the top 5 states with the highest & lowest average delivery time.

i) top 5 states with the highest average delivery time.

```
      SELECT
 DISTINCT
 c.customer_state,
 AVG(DATE_DIFF(o.order_delivered_customer_date,o.order_purchase_timestamp, DAY))
OVER (PARTITION BY c.customer_state) AS
  avg_delivery_time,
FROM `target.orders` AS o
JOIN `target.customers` AS c
ON o.customer_id = c.customer_id
ORDER BY avg_delivery_time DESC
LIMIT 5
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETA

Row	customer_state	avg_delivery_time
1	RR	28.97560975609...
2	AP	26.73134328358...
3	AM	25.98620689655...
4	AL	24.04030226700...
5	PA	23.31606765327...

i) top 5 states with the lowest average delivery time.

```
SELECT
 DISTINCT
 c.customer_state,
 AVG(DATE_DIFF(o.order_delivered_customer_date,o.order_purchase_timestamp, DAY))
OVER (PARTITION BY c.customer_state) AS
  avg_delivery_time,
FROM `target.orders` AS o
JOIN `target.customers` AS c
ON o.customer_id = c.customer_id
ORDER BY avg_delivery_time ASC
LIMIT 5
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	customer_state	avg_delivery_time
1	SP	8.298061489072...
2	PR	11.52671135486...
3	MG	11.54381329810...
4	DF	12.50913461538...
5	SC	14.47956019171...

D. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

```
      SELECT

 c.customer_state,
 (AVG(DATE_DIFF(o.order_delivered_customer_date,o.order_purchase_timestamp, DAY))-
AVG(DATE_DIFF(o.order_estimated_delivery_date,
 o.order_delivered_customer_date, DAY)))  AS delivery_time
FROM `target.orders` AS o
JOIN `target.customers` AS c
on o.customer_id = c.customer_id
WHERE order_status = "delivered" AND o.order_delivered_customer_date IS NOT NULL
AND o.order_purchase_timestamp IS NOT NULL AND o.order_estimated_delivery_date IS
NOT NULL AND o.order_delivered_customer_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY delivery_time
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CI

Row	customer_state	delivery_time
1	SP	-1.83639551538...
2	PR	-0.83749746089...
3	MG	-0.75691386295...
4	RO	-0.21810699588...
5	AC	0.874999999999...
6	DF	1.390384615384...
7	RS	1.837387724550...
8	SC	3.874224478285...
9	GO	3.883495145631...
10	RJ	3.952631578947...

Key findings: The output shows the delivery time spread across each state.

VI. Analysis based on the payments:

A. Find the month on month no. of orders placed using different payment types.

First approach:

```
SELECT
 DISTINCT
 p.payment_type,
 COUNT(o.order_id) OVER (PARTITION BY p.payment_type) AS num_of_orders
FROM `target.payments` AS p
JOIN `target.orders` AS o
ON p.order_id = o.order_id
WHERE o.order_status = "delivered"
GROUP BY p.payment_type,o.order_id
ORDER BY p.payment_type
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	payment_type	num_of_orders
1	UPI	19191
2	credit_card	74304
3	debit_card	1485
4	voucher	3679

Key findings: The output shows the total number of order spread across each payment type.

Second approach:

```
SELECT
 DISTINCT
 EXTRACT(MONTH FROM o.order_purchase_timestamp)  AS month,
 p.payment_type,
 COUNT(o.order_id) OVER (PARTITION BY p.payment_type,EXTRACT(MONTH FROM
o.order_purchase_timestamp)) AS num_of_orders
FROM `target.payments` AS p
JOIN `target.orders` AS o
ON p.order_id = o.order_id
WHERE o.order_status = "delivered"
GROUP BY o.order_purchase_timestamp,month,o.order_id,p.payment_type
ORDER BY p.payment_type,month
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | CHART

Row	month	payment_type	num_of_orders
1	1	UPI	1661
2	2	UPI	1665
3	3	UPI	1881
4	4	UPI	1739
5	5	UPI	1982
6	6	UPI	1778
7	7	UPI	2011
8	8	UPI	2021
9	9	UPI	868
10	10	UPI	1006

Key findings: The approach shows the total number of order spread across each payment type across each month.

B. Find the no. of orders placed on the basis of the payment instalments that have been paid.

```
SELECT COUNT(DISTINCT order_id) AS num_of_orders, COUNT (payment_value) AS
num_of_installments FROM `ecommerce-394317.target.payments`  WHERE payment_value!=
0 AND payment_installments >= 1 GROUP BY payment_installments
```

Query results

JOB INFORMATION | RESULTS | JSON | EX

Row	num_of_orders	num_of_installments
1	49057	52537
2	12389	12413
3	10443	10461
4	7088	7098
5	5234	5239
6	3916	3920
7	1623	1626
8	4253	4268
9	644	644
10	5315	5328

Key findings: The output shows the number of orders placed over spread across number of instalments.